Plectra: Scheduling Parallelism for Diffusion Transformer Serving

Xinwei Qiang
qiangxinwei@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Yifan Hu
hu_y_f@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Shixuan Sun
sunshixuan@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Jing Yang
jyang23@gzu.edu.cn
Guizhou University
Guiyang, China

Han Zhao
zhao-han@cs.sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Chen Chen
chen-chen@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Yu Feng
y-feng@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Jingwen Leng
leng-jw@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Minyi Guo
guo-my@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Abstract

Diffusion Transformers (DiTs) have become the dominant architecture for image and video generation, creating growing demand for efficient DiT serving. Existing systems assign each request a fixed parallel configuration throughout its lifetime. However, DiT workloads exhibit substantial heterogeneity across requests, execution stages, and system conditions, making static parallelism inefficient and often leading to poor GPU utilization and degraded service quality.

This paper argues that DiT serving should treat GPU parallelism as a first-class schedulable resource. We present Plectra, a policy-programmable runtime for elastic DiT serving that dynamically adapts the parallelism of running requests to workload demands and service objectives. Plectra decomposes requests into independently schedulable trajectory tasks for online GPU reallocation, and proposes group-free collectives, a lightweight communication abstraction that supports low-overhead online formation and reconfiguration of arbitrary execution groups.

We implement Plectra in vLLM-Omni and evaluate it on representative image and video diffusion workloads. Compared with fixed-pipeline execution with static parallelism, Plectra's earliest-deadline-first (EDF) best-fit policy improves throughput by up to 6.01×, reduces mean latency by up to 95%, and lowers SLO violation rates by up to 90%, while group-free collectives reduce communication-group setup overhead from 778 ms to approximately 60 μs.

Our code is available at https://github.com/SJTU-Liquid/GF-DiT.

1 Introduction

DiTs [31] have emerged as the dominant architecture for modern generative workloads, enabling high-quality text-to-image, text-to-video, and multimodal content generation [6, 13–15, 19, 22, 26, 32, 41, 44, 48, 51, 52, 54]. Serving DiT models efficiently has become a critical systems challenge.

Unlike large language models (LLMs), DiT inference consists of three stages: an *encoder* that transforms prompts into embeddings, a compute-intensive *denoising* stage that iteratively refines latent states through tens of diffusion steps, and a *decoder* that generates the final image or video. Denoising dominates execution time and resource consumption.

To serve DiT workloads, recent systems such as vLLM-Omni [50] and SGLang Diffusion [35] adopt a fixed multi-GPU execution strategy. As illustrated in Fig. 1, each request is assigned a predetermined parallel configuration, and all three stages execute atomically under the same resource allocation. DiT workloads, however, are highly heterogeneous: output resolution, video duration, and denoising-step count vary by orders of magnitude across requests, and within a request the encoder and decoder stages are lightweight while denoising can effectively utilize many more GPUs.

Fixed-pipeline execution with static parallelism therefore suffers two fundamental limitations: long-running requests cause head-of-line (HoL) blocking that delays shorter ones, and a single static configuration wastes GPUs on lightweight stages while under-provisioning large denoising workloads. Existing systems consequently struggle to achieve high SLO attainment and high GPU utilization at the same time.

At the root of both limitations is static parallelism: once a request is assigned a parallel configuration, its GPU allocation never changes, so the system cannot react to arrivals, queue conditions, or evolving service objectives. A request appropriately provisioned at admission may become over- or under-provisioned as conditions evolve.

These observations motivate a different perspective: instead of treating parallelism as a fixed execution configuration, a DiT serving system should treat parallelism as a

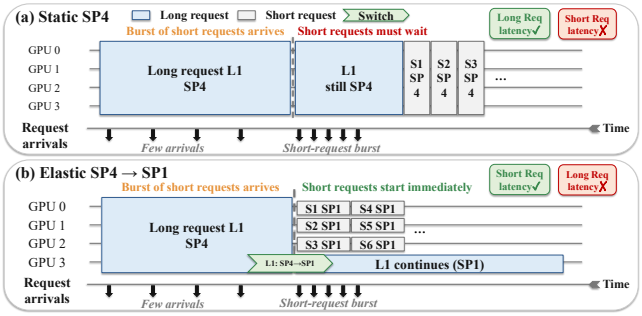


Figure 1. Elastic parallelism exposes policy tradeoffs. Static SP4 preserves the long request's latency but makes short requests wait; shrinking it to SP1 admits short requests sooner at the cost of delaying the long request.

first-class schedulable resource. Just as operating systems allocate CPU time among competing processes, a DiT serving runtime should dynamically allocate GPU parallelism among competing requests according to workload demands and service objectives; existing systems schedule requests, whereas a DiT runtime should additionally schedule parallelism.

Realizing this vision is challenging: the runtime must repeatedly reconfigure resource allocations among concurrently executing requests with minimal disruption, and different deployments optimize for different objectives, including latency, throughput, fairness, and cost. The challenge is thus not only efficient runtime reconfiguration but a programmable substrate upon which diverse scheduling policies can be built.

Our Work. To address these challenges, we present **Plectra**, a policy-programmable runtime for elastic DiT serving. Plectra is enabled by two unique properties of DiT workloads. First, DiT execution maintains a *lightweight execution state*: stages and denoising steps exchange mainly latent states and embeddings of only several to tens of MBs. Second, DiT requests exhibit a *predictable execution structure*: the denoising trajectory is largely determined before execution begins, enabling accurate estimation of execution costs and future resource demands. Together, these properties make DiT serving uniquely amenable to online parallelism adaptation.

Building upon these insights, Plectra consists of three key components. First, an *asynchronous execution abstraction and runtime model* decomposes a DiT request into independently schedulable trajectory tasks, including encoding, decoding, and individual denoising steps, so that every task boundary is an explicit rescheduling point and parallelism becomes a runtime-managed resource.

Second, elastic parallelism requires execution groups to be created and reconfigured online. We introduce *group-free collectives*, a lightweight communication abstraction that eliminates expensive communicator construction and supports arbitrary GPU-group formation through lightweight logical descriptors, reducing reconfiguration overheads to a negligible level.

Finally, runtime optimizations such as layout-aware artifact migration and simulation-driven policy optimization, combined with Plectra's programmable policy interface, let users rapidly evaluate and deploy customized scheduling strategies on the same runtime substrate.

We implement Plectra in vLLM-Omni and evaluate it on image and video diffusion workloads, where one runtime substrate supports throughput-oriented, latency-oriented, SLO-aware, and custom policies. Compared with fixed-pipeline execution with static parallelism, Plectra's Earliest Deadline First (EDF) best-fit policy improves throughput by up to 6.01×, reduces mean latency by up to 95%, and lowers SLO violation rates by up to 90%; group-free collectives reduce communication-group setup overhead from up to 778 ms to approximately 60 μs. In summary, this paper makes the following contributions:

- We introduce *parallelism as a first-class schedulable resource*, a new abstraction for adaptive DiT serving, and design **Plectra**, a policy-programmable runtime that realizes it through asynchronous execution and reschedulable trajectory tasks.
- We propose *group-free collectives*, which make dynamic execution-group formation practical by eliminating expensive communicator construction.
- We implement Plectra and demonstrate significant improvements over fixed-pipeline execution with static parallelism.

2 Background and Motivation

This section presents the observations that motivate Plectra: DiT workloads expose rescheduling boundaries throughout execution, their execution structure is largely predictable, and no single parallel configuration is optimal across stages, requests, and system conditions.

2.1 Rescheduling Boundaries

In autoregressive LLM serving, each generated token produces a valid continuation state, and chunked prefill incrementally constructs reusable KV caches [2]; these token-sequence boundaries enable batching, preemption, and request scheduling. DiT execution follows a different structure. During denoising, the model repeatedly refines a latent state along a *diffusion trajectory*, i.e., the sequence of latent states traversed from the initial noisy latent to the final clean latent. Each denoising step performs bidirectional computation over the entire latent sequence, so a latent-token prefix does not represent semantically complete progress; internal boundaries introduced by sequence parallelism or model-specific partitioning are implementation artifacts rather than portable scheduling points.

Instead, DiT requests expose natural scheduling opportunities at stage and denoising-step boundaries. A request first runs conditioning encoders to produce embeddings and

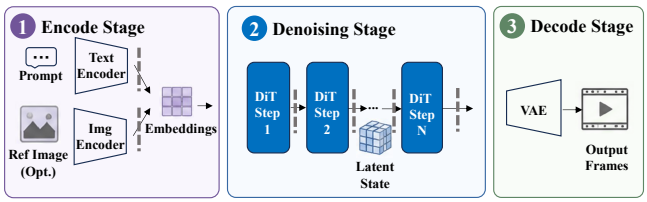


Figure 2. Structure of a diffusion serving request. Encoding produces conditioning embeddings, denoising iteratively advances the diffusion trajectory, and VAE decoding generates the final output. Trajectory task boundaries provide semantically valid rescheduling points for runtime adaptation.

initialize latent states; the denoising stage then applies the DiT model across a sequence of timesteps; finally, a VAE [18] decoder converts the denoised latent into the output image or video.

As illustrated in Fig. 2, completing an encoder stage, a decoder stage, or a denoising step produces a semantically complete execution state that can be safely transferred, resumed, or executed under a different resource configuration. Each such point is a *rescheduling boundary*, and Plectra represents the stage or step between boundaries as an independently schedulable *trajectory task*, creating opportunities for online parallelism adaptation while preserving request semantics.

2.2 Predictable Execution Structure

Much of a request's execution structure is known before execution begins: request parameters or service defaults fix the output resolution, video duration, and denoising-step count at admission, which for a given model determine the latent sequence length and the diffusion trajectory. Once a request is decomposed into trajectory tasks, its execution graph is largely deterministic: the number of tasks, their dependencies, and the remaining path are known a priori. Each task runs a known stage or step on a known input shape, so its cost under a candidate parallel configuration can be estimated from profiling data. This predictability distinguishes DiT serving from many dynamic workloads: the runtime can reason about alternative allocations before applying them, which together with trajectory task boundaries makes online parallelism adaptation practical.

2.3 Why Parallelism Should Be Schedulable

We next use microbenchmarks and trace-driven replay to show that the optimal degree of parallelism depends on the execution stage, request characteristics, and system conditions.

Stage-level heterogeneity. Fig. 3(a) reports stage latencies under different execution-group sizes. Denoising benefits from larger sequence-parallel groups because it dominates computation and operates on long latent sequences, whereas text encoding remains effectively single-rank and VAE decoding scales differently. A single request-wide configuration thus either wastes resources on lightweight stages or under-provisions denoising.

Shape-dependent parallelism. Fig. 3(b) reports denoising-task latency, normalized to single-GPU execution, across request shapes and sequence-parallel degrees. Larger requests expose more computation per step and benefit more from additional GPUs, while smaller requests fail to amortize communication overheads. Even for the same model and stage, the optimal configuration depends strongly on request shape.

System-dependent parallelism. Fig. 3(c) evaluates serving configurations using trace-driven replay. Under light load, allocating more GPUs per request minimizes latency; under heavy load, reducing per-request parallelism improves throughput and SLO attainment by increasing concurrency. As workload composition and objectives evolve, the preferred configuration changes accordingly.

No single parallel configuration is therefore optimal throughout a request's lifetime, motivating our central design principle: a DiT serving system should treat *GPU parallelism as a first-class schedulable resource*. Trajectory boundaries create opportunities for adaptation, predictable execution makes it feasible, and workload heterogeneity makes it necessary. Enabling such adaptation efficiently motivates the asynchronous execution abstraction, group-free collectives, and policy-programmable runtime presented in the remainder of the paper.

3 Plectra Abstraction and Overview

Plectra exposes runtime-visible trajectory tasks along each diffusion trajectory, lets a policy bind each ready task to an execution layout, and realizes the resulting decisions through an asynchronous runtime. This section defines the abstraction; Sections 4 and 5 describe the communication mechanism and runtime implementation.

3.1 Reschedulable Trajectory Tasks

Plectra represents each diffusion request as a placement-agnostic *trajectory task graph* whose nodes are independently schedulable trajectory tasks and whose edges are artifact dependencies. A trajectory task may represent a model stage, such as encoding, latent preparation, or decoding, or a single denoising step. Completing a task produces a well-defined model state, so the runtime may reconsider task order, placement, and parallelism at the next boundary without exposing partially updated model internals.

Trajectory tasks communicate through *logical artifacts*: conditioning features, latent state, scheduler metadata, decoded outputs, or other model-specific state. An artifact records a dependency and semantic role but not a physical layout; it may later be materialized as replicated tensors,

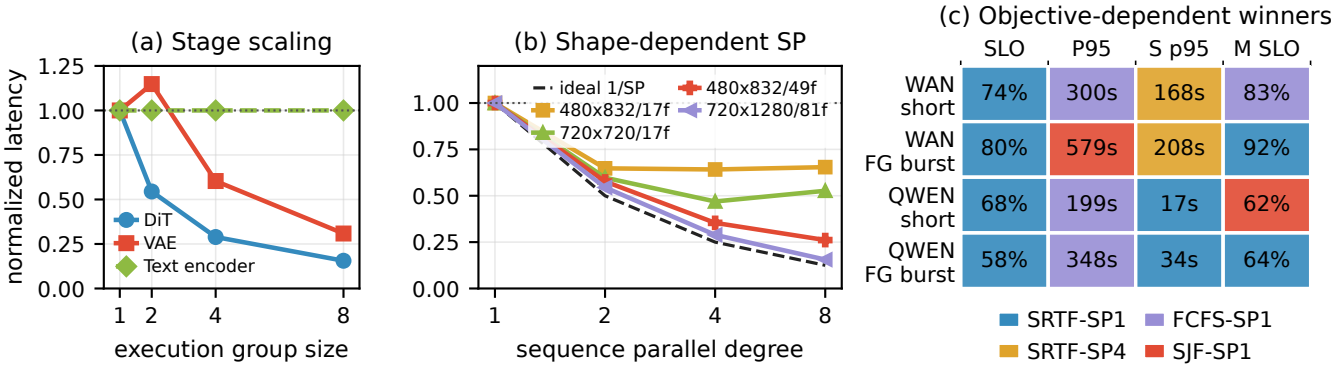


Figure 3. Motivating measurements for elastic DiT serving. (a) Different stages exhibit distinct scaling behavior and resource preferences. (b) The performance benefit of parallelism depends on request shape. (c) Different workload conditions favor different parallelism choices, indicating that no single parallel configuration is universally optimal.

sequence-parallel shards, or model-specific metadata according to the layouts of its producer and consumer.

The graph exposes two properties to the runtime: task boundaries are explicit rescheduling points at which GPU allocation can change, and the graph and request shape are largely known at admission, so the runtime can estimate remaining work and compare candidate configurations before a task executes.

3.2 Execution Layouts and Policy Interface

A Plectra policy schedules a ready trajectory task by assigning it an *execution layout*. An execution layout consists of an ordered logical execution group and a parallel specification describing how the task uses that group. For example, a DiT task may use a multi-rank sequence-parallel layout, whereas a lightweight encoder task may use a single rank.

At each scheduling point, the policy observes ready tasks, request metadata, resource availability, optional deadlines or priorities, and cost estimates for candidate layouts, and returns (*task*, *execution layout*) dispatch decisions. This interface lets policies jointly control task order, placement, and parallelism: a policy may prioritize short requests, allocate additional ranks to an urgent request, or shrink a running request at its next trajectory boundary to admit new work. The policy operates only on logical tasks, artifacts, and execution groups; the runtime validates each decision and handles execution mechanics. In particular, group-free collectives make policy-selected logical groups executable without serving-path communicator construction, and layout-aware artifact migration reconstructs artifacts when consecutive tasks use different layouts.

3.3 System Overview

Fig. 4 shows how the abstraction maps to the system. A model-specific adapter converts each incoming request into trajectory tasks and logical artifacts. The control plane maintains graph dependencies, artifact metadata, resource state,

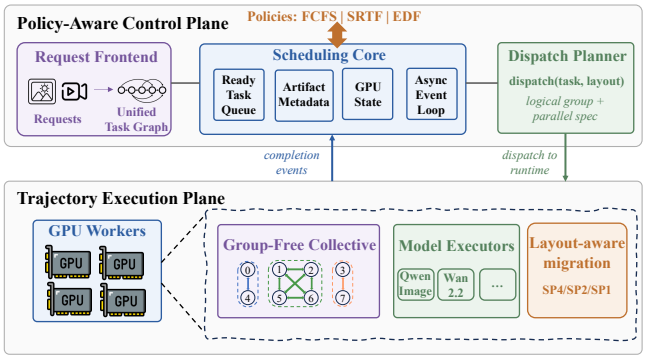


Figure 4. Plectra system overview. Incoming requests are converted into trajectory task graphs, scheduled by the control plane through a programmable policy interface, and dispatched to workers under dynamic execution layouts. Group-free collectives and layout-aware artifact migration make these dynamic layout choices executable at runtime.

and policy-visible request information; when tasks become ready or resources are released, it invokes the policy and dispatches selected tasks under their assigned layouts. The execution plane consists of GPU workers, model executors, and communication runtimes. Workers execute tasks asynchronously and report progress back, letting scheduling and CPU-side preparation overlap with GPU execution.

The same control-plane abstraction supports offline policy exploration: the simulator replaces GPU workers with completion events from profiled task costs. Because both backends preserve the same task graph, resource state, and policy interface, policies evaluated in simulation can be deployed without rewriting their decision logic.

4 Group-Free Collectives

The abstraction in Section 3 lets policies assign trajectory tasks to dynamic execution layouts. DiT tasks often need subgroup collectives within those layouts, such as all-gather or all-to-all operations used by sequence-parallel execution [50].

Table 1. NCCL subgroup setup costs on an 8-GPU server.

Group size	`new_group` (ms)	First coll. (ms)	Warm coll. (ms)	Memory (MB/GPU)
2	0.77	217.3	0.19	484
4	0.46	532.0	0.56	535
6	0.57	663.5	0.72	535
8	0.57	777.7	0.89	535

Plectra executes these operations with group-free collectives (GFC). A conventional communicator binds collective state to one fixed rank set, so every newly selected group pays cold-start initialization and per-group memory (Table 1), and pre-creating candidate groups is combinatorial. GFC instead pays one world-level setup cost at initialization, registers each dynamic subgroup as lightweight metadata in about 60 μs, and uses a per-edge agreement protocol that keeps overlapping groups correct without per-group communicator state.

4.1 Limits of Process-Group-Based Collectives

Process groups work well for static model-parallel deployments, where the same tensor- or sequence-parallel groups persist for the lifetime of the server. Policy-programmable serving breaks this assumption: a throughput policy may run many requests on small groups, a deadline-aware policy may temporarily enlarge a group for an urgent request, and a single request may use different groups at different points on its trajectory. These subgroup rank sets are chosen online rather than fixed at startup.

Creating a process group per selected rank set is a poor fit for the serving path: construction involves host-side coordination and communicator initialization, adding latency exactly when the scheduler makes a fine-grained placement decision. Pre-creating candidates instead grows combinatorially. Covering only size-2 and size-4 groups already requires $\binom{8}{2} + \binom{8}{4} = 98$ process groups on an 8-GPU server and $\binom{72}{2} + \binom{72}{4} = 1{,}031{,}346$ on an NVL72 domain, which is impractical given the per-group memory measured below. Table 1 quantifies the cost on an 8-GPU server: the `new_group` call itself is sub-millisecond, but the first collective on a subgroup triggers hundreds of milliseconds of cold-path initialization and reserves roughly 0.5 GB of GPU memory per group. Warm collectives are fast; the problem is the one-time setup and memory that dynamic policies would pay for each newly selected rank set.

Plectra therefore separates execution groups from communicators: a policy chooses a logical execution group as part of a task's layout, and the communication runtime executes collectives for that group using GFC descriptors and world-level communication state.

4.2 Scope and Ordering Assumptions

GFC targets a controlled serving setting rather than arbitrary user-issued collective communication: the control plane decides which trajectory tasks run, which ranks participate in each layout, and which subgroup collectives those tasks issue. Executors invoke collectives through the runtime interface, allowing Plectra to support dynamic subgroups without exposing a general distributed communicator API.

The main correctness requirement is ordering. A rank may participate in many logical groups over time, and these groups may overlap: ranks 0 and 1 may first communicate in group $\{0, 1, 2, 3\}$ and later in group $\{0, 1\}$. If rank 0 enters these shared collective instances in a different order from rank 1, a synchronization signal from one instance may be mistaken for another, causing incorrect data movement or deadlock [29].

Plectra therefore assumes *pairwise-consistent ordering*: for any pair of ranks, the collective instances in which both ranks participate are issued in the same order at both ranks. This assumption is weaker than requiring a single global order over all collectives. Two disjoint groups may proceed independently, and groups that overlap only partially need only agree on the relative order observed by each shared rank pair. Under this runtime model, Plectra enforces pairwise-consistent ordering through centralized scheduling and ordered submission. The scheduler is the only component that creates dynamic execution layouts, and each worker submits collectives for assigned tasks to a local ordered communication stream. Thus, any two ranks that appear together in multiple groups observe the same order of their shared collective instances.

4.3 Symmetric Buffers and Logical Groups

Group-free collectives separate communication state from subgroup membership. At initialization, Plectra performs a single world-level setup in which all ranks allocate and register symmetric communication buffers [33], a common addressable substrate for data movement and synchronization paid for once at server start.

At runtime, a subgroup is a logical group descriptor: the ordered participating ranks, a runtime group identifier, and the local rank's position. Creating a logical group is a metadata operation: the runtime constructs no communicator, allocates no per-group communication state, and requires no participation from non-members. A trajectory task can use a logical group once the participating ranks receive the same descriptor from the control plane. In our implementation, registration takes approximately **60** μs: CPU-side bookkeeping plus one host-to-device copy of the participating rank IDs. The descriptor gives each rank the information needed to interpret collective semantics inside the group, such as the local rank index used by a sequence-parallel executor and the mapping from group-local peers to global ranks, while the

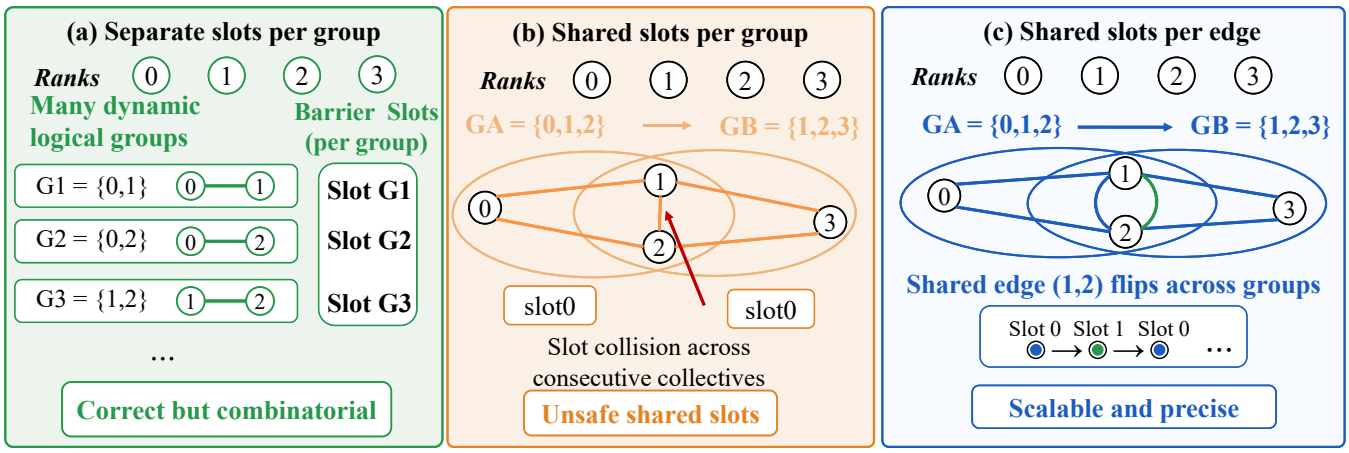


Figure 5. Signal-state designs for dynamic overlapping groups. (a) Separate slots for every logical group avoid collisions but require combinatorial per-group state. (b) Sharing slots across groups allows consecutive overlapping collectives to collide. (c) Plectra assigns double-buffered phase state to each rank edge, so a shared edge flips slots consistently across groups without allocating per-group signal state.

symmetric buffers provide storage and addressing. Together they replace the role of a per-subgroup process group.

Executors operate on ordinary tensors: when a collective is issued, the runtime stages data into a symmetric buffer region, performs subgroup data movement, and copies results back. Staging hides GFC from model code and lets the runtime choose different backends for the same logical group.

4.4 Edge-Based Agreement for Overlapping Groups

Logical group descriptors and symmetric buffers make subgroup communication addressable, but they do not establish agreement on collective instances. Since logical groups can overlap, the same pair of ranks may communicate together in different groups over time. The runtime must ensure that a signal written by one rank for one collective is interpreted by its peer as belonging to that same collective.

Plectra identifies each collective instance with a synchronization token derived from the runtime session, the logical group identity, and a per-group logical epoch. The epoch advances whenever the group issues a new collective. The token therefore tells a peer which collective instance a signal belongs to, even if the same signal slot is reused later.

The remaining problem is deciding when a signal slot can be reused. A single slot per sender-receiver pair is unsafe: a rank could publish the next token before its peer has observed the previous one, overwriting the only evidence of arrival. Double buffering fixes this local lifetime problem by alternating between two slots, so instance $N + 1$ does not overwrite the token for instance N. However, the phase bit that selects the slot must advance in the same way at both endpoints for exactly the collective instances they share.

Group-level slot designs are unsatisfactory (Fig. 5): per-group slots are correct but combinatorial in the number of possible groups, while a shared slot pool is compact but unsafe: consecutive overlapping groups can make their shared

Algorithm 1: Per-edge flip agreement for a GFC collective

Input: Logical group G, local rank $r \in G$, collective instance c

State: each local edge (r, p) keeps a one-bit phase $\phi_{r,p}$ and two signal slots;

Invariant: for every rank pair, shared collective instances are submitted in the same order at both endpoints;

$\tau \leftarrow \textsc{Token}(G, c)$;

foreach $p \in G \setminus \{r\}$ **do**

  $e \leftarrow (r, p)$;

  $s[p] \leftarrow \phi_{r,p}$;

  $\phi_{r,p} \leftarrow 1 - \phi_{r,p}$; // Flip phase

  $\textsc{Publish}(e, s[p], \tau)$;

foreach $p \in G \setminus \{r\}$ **do**

  $e' \leftarrow (p, r)$;

  wait until $\textsc{Observed}(e', s[p], \tau)$;

rank pair publish to the same slot before the earlier token is consumed. The slot sequence must be owned by the entity that actually observes reuse: the rank pair.

Plectra therefore attaches the phase bit to each ordered rank edge. A group barrier requires each rank to agree with every other rank in the logical group and completes only after all pairwise edges observe the expected token. Because slot ownership follows rank pairs, unrelated groups cannot overwrite one another, and overlapping groups interact only through the edges whose endpoints actually need to agree.

Algorithm 1 hides the signal layout. In the implementation, each ordered rank pair owns two signal slots: a rank

publishes to the peer's incoming slot and waits on the reciprocal slot, selecting the slot by the parity of a local per-edge sequence counter. Because the control plane submits pair-sharing collectives in the same order at both endpoints (Section 4.2), the two ranks select matching slots for the same collective instance without exchanging slot state.

The protocol needs no separate acknowledgement phase. Since each rank submits edge instances on one ordered communication stream, a slot used by edge instance N is not reused until $N + 2$; before a rank can publish $N + 2$, its local $N + 1$ must have returned, which implies the peer published $N + 1$, which in turn happens only after the peer consumed the token for N. Double buffering therefore suffices to prevent a token from being overwritten before its peer observes it. Publish and observe use system-scope release and acquire operations, so data movement begins only after all pairwise arrivals are visible.

4.5 Backend-Aware Collective Execution

After a collective instance reaches agreement, Plectra executes data movement with a backend chosen for the message size and pattern: CUDA-kernel communication has low setup overhead and can fuse simple packing or reduction logic; copy-engine transfers reduce SM interference for larger point-to-point payloads; and TMA-style transfers, when available, improve bulk movement of structured tensor tiles. A selector populated from microbenchmark results chooses the backend and chunk size for each collective pattern and message-size range.

Requiring all model intermediates to live in a special symmetric-memory pool would make integration intrusive, so executor tensors stay ordinary and the runtime stages each collective through symmetric buffer regions: it copies the local input slice in, moves data remotely through the selected backend, and writes results back to the executor-visible tensor. Staging adds local copy work, but the payload is divided into chunks so staging of one chunk overlaps with remote transfer of another, hiding much of the copy overhead. All of this sits behind the GFC API: the scheduler picks a task's layout, the executor issues the collectives its parallel specification requires, and the communication runtime handles agreement, staging, backend selection, and data movement.

5 Plectra Implementation

We implement Plectra in vLLM-Omni as an event-driven control plane and a set of execution-plane components.

5.1 Event-Driven Control and Execution Planes

Plectra separates scheduling state from task execution. The control plane owns request admission, trajectory task graphs, dependency state, artifact metadata, resource availability, and policy invocation; the execution plane contains worker processes, model executors, and communication runtimes.

The key control-path event is dispatch completion. When the control plane dispatches a task, the runtime resolves its logical execution group, prepares inputs, allocates output artifact handles with their expected layouts, and sends the task to participating workers. Once this CPU-side dispatch finishes, the control plane can prepare successor state and issue independent work while the GPU task is still running. Device execution is tracked separately: workers record CUDA events around task execution, and a monitor thread reports execution-start and completion events that materialize output artifacts, release worker resources, detect failures, and calibrate the runtime cost model with measured durations. Consumers block until their inputs are materialized, but scheduling-state preparation never waits for device completion. Policy evaluation, dependency updates, and migration planning overlap with GPU execution instead of extending the critical path at every trajectory boundary.

Failure handling. On a task-level error, the control plane fails the request, releases its resources, and notifies the policy; a GPU or worker crash requires a runtime restart, as deployments often restart the node. GFC permits finer-grained recovery: in a prototype, survivors poll a dead-peer mask in the edge barrier, abort pending waits, and regroup.

5.2 Model Adapters

Plectra keeps model-specific logic behind a narrow adapter interface with three components. The *request converter* determines the task granularity exposed to scheduling, emitting stage tasks for components such as text encoding and VAE decoding plus one task per denoising step. *Task executors* accept a task description and execution layout, invoke the corresponding model stage, and use the communication runtime when the parallel specification requires collectives. *Artifact codecs* connect model-specific tensor structures to the placement-agnostic graph: for each logical artifact, a codec identifies replicated, sharded, and metadata-only fields and reports the global shape and per-rank slice under a given layout, which the runtime uses to decide whether an artifact can be consumed directly or requires migration. A policy therefore sees ready tasks, request metadata, resource availability, and cost estimates, but not how latents are sharded or how a stage invokes collectives, so adding a new diffusion pipeline only requires implementing an adapter.

5.3 Layout-Aware Artifact Migration

When adjacent trajectory tasks use different execution layouts, the runtime reconstructs their logical artifacts without exposing migration decisions to the policy, in three steps. First, the source-group leader obtains source and destination views of each migrated artifact from the codecs. Second, it derives a migration plan: for each sharded tensor field, the planner intersects each source-owned slice with each

destination-required slice, and every non-empty intersection becomes a transfer entry (source rank, destination rank, tensor ranges, byte size). Third, it distributes the plan; each rank packs its local tensor ranges, exchanges data with peers point-to-point, and installs received ranges into the destination representation, after which the consumer reads the artifact under its assigned layout.

Migration transfers run through the same group-free substrate, using a logical pair group and the agreement mechanism from Section 4 for each migration edge. This avoids PyTorch [4] point-to-point paths that may silently construct two-rank communication groups in the background.

5.4 Scheduling Policies

Plectra exposes scheduling as a policy-layer decision. A policy observes ready trajectory tasks, request metadata, resource availability, and cost estimates, then returns a task ordering and execution layout for each dispatched task. We implement three representative policies.

First-Come, First-Served (FCFS) with workload-aware group assignment partitions the cluster into worker groups and serves requests in arrival order, assigning each ready task to the feasible group with the lowest estimated queued workload, combining simple request ordering with runtime-managed load balancing.

Shortest Remaining Time First (SRTF) with per-rank local queues assigns each request to a feasible rank by estimated queued workload; each rank then schedules its ready tasks by shortest remaining trajectory work [34], estimated from request shape, trajectory position, and profiled task costs, exploiting diffusion's predictable costs to reduce mean latency. Ranking is re-evaluated at every step boundary, so short arrivals preempt long requests; Shortest Job First (SJF) is a non-preemptive variant.

Earliest Deadline First (EDF) with best-fit parallelism targets SLO settings: it orders ready tasks by deadline [21] and uses the cost model to select the smallest parallel configuration predicted to meet the deadline, growing a request's group at the next trajectory boundary when it risks missing its deadline. This provides SLO-aware dynamic placement without hard-coding deadline logic into model executors.

These policies are deliberately simple: they differ only in task ranking and layout choice. Dependency tracking, asynchronous dispatch, dynamic-group communication, and artifact migration are handled by the runtime.

5.5 Simulator and Cost Model

Since the preferred policy depends on workload mix, request shapes, objective, and available ranks (Section 2.3), a deployment may need to compare several policies before choosing one. Exploiting the predictable structure from Section 2.2, the simulator replaces worker execution with cost-model completion events while preserving task readiness, dependency updates, resource allocation, and policy invocation; its cost model is populated from profiled task latencies indexed by model, task type, request shape, and parallel configuration. Because the simulator and the online runtime share the same policy interface, a policy selected offline can be deployed without rewriting its decision logic. The simulator is therefore an alternative execution backend for the same trajectory abstraction rather than a separate scheduling model.

6 Evaluation

We evaluate Plectra on image and video diffusion serving workloads.

6.1 Experimental Setup

We implement Plectra on vLLM-Omni 0.17.0 [50] using Python 3.12, CUDA 12.9, PyTorch 2.10 [4], Triton 3.6.0 [42], and NCCL 2.27.5 [29]. Experiments run on a 4-GPU H20 server and an 8-GPU A100 server, using Wan2.2-5B [44] for video generation and Qwen-Image [46] for image generation.

For each model and platform, we generate fixed-duration request traces with arrival rates calibrated from measured platform throughput, so policies are compared under comparable serving pressure. Fig. 7 shows the two workloads: a compact mixed-arrival period and a foreground-burst setting where bursts of short requests arrive while longer requests may already be in flight. Together, they test whether the runtime can preserve mixed-request concurrency while giving urgent short requests priority in scheduling. Short/medium/long (S/M/L) requests generate 480×832/49-frame, 480×832/81-frame, and 720×1280/81-frame videos for Wan2.2, and 512×512, 1024×1024, and 1536×1536 images for Qwen-Image.

Each request's SLO is defined relative to its profiled standalone service time: a class-c request arriving at time a receives a scheduler-visible deadline $a + \alpha_c T_c$, where T_c is the profiled service time of its request shape (re-profiled per platform). We fix $\alpha_{\mathrm{S/M/L}} = 2.0/2.5/3.5$ for Wan2.2 and $1.5/2.0/6.0$ for Qwen-Image before comparing policies and do not tune them per policy; the client-visible deadline adds a fixed allowance for non-profiled overheads (5 s for Wan2.2, 1 s for Qwen-Image). SLO attainment is the fraction of submitted requests completing by this deadline. Requests also carry a loose client-side timeout that bounds experiment duration under overload: 1500 s for image and 3600 s for video, more than 3× the largest deadlines (471 s and 942 s). Timed-out requests have therefore already missed their deadlines; they count as failures and SLO violations. Latency statistics include only completed requests and throughput counts completed requests per second, so latency results for policies with failures are optimistic.

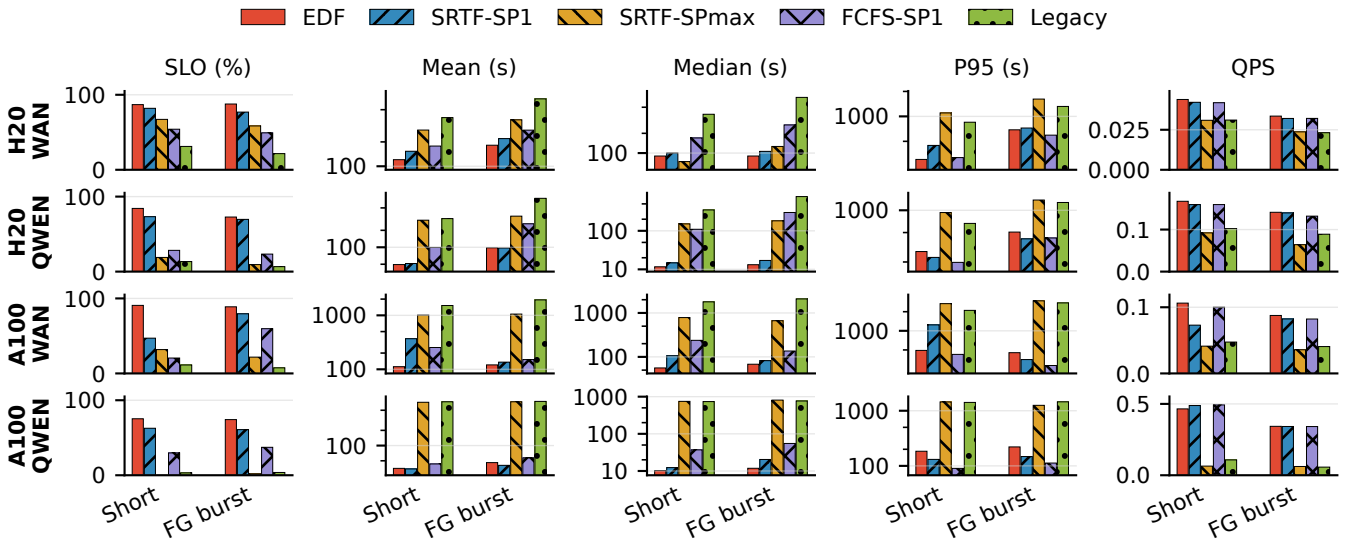


Figure 6. End-to-end serving results on H20 and A100. Each row is a platform-model pair, and each column reports one metric across the short and foreground-burst workloads. Legacy is the native vLLM-Omni fixed-pipeline path with static parallelism; other policies run on Plectra. SLO attainment counts failed requests as violations; latency statistics cover completed requests only.

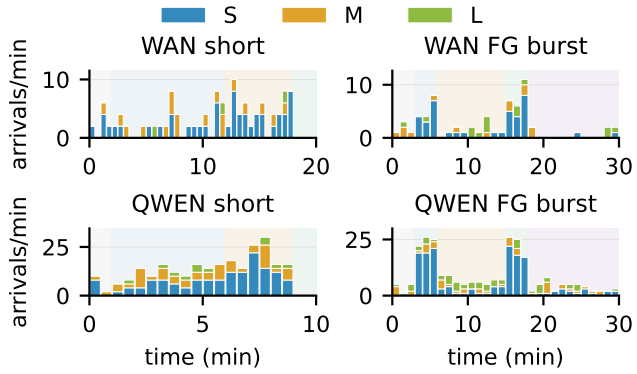


Figure 7. Arrival-rate timelines of the two main-result workloads; colors denote short (S), medium (M), and long (L) request classes.

6.2 Baselines

We compare against the native vLLM-Omni serving path (Legacy): its original fixed-pipeline execution with static parallelism over the full machine. EDF, SRTF-SP1, SRTF-SPmax, and FCFS-SP1 are implemented on the Plectra policy interface (Section 5.4). Recent stage- or step-level DiT serving systems target different settings: GenServe [49] co-serves heterogeneous models, and TridentServe [47] allocates stage-level resource pools at cluster scale; both are orthogonal to the single-model, intra-node setting studied here. TetriServe [23] is the closest system. Its released implementation targets earlier-generation models, FLUX.1 [5] and Stable Diffusion 3 [9], whereas we evaluate newer Wan2.2 and Qwen-Image models. To compare its scheduler on these models, we port its scheduling algorithm to our policy interface and evaluate it in our simulator under identical traces and profiled costs (Section 6.9).

6.3 Main Results

Fig. 6 shows that different objectives favor different policies. EDF achieves the highest SLO attainment in every platform-model-workload setting, with large gains in bursty cases: on A100 Wan2.2 foreground burst, SLO attainment improves from 7.5% with Legacy to 88.8% with EDF; on H20 Qwen-Image foreground burst, from 6.6% to 72.8%. Across all main-result settings, EDF improves throughput by up to 6.01×, reduces mean latency by up to 95.2%, and reduces SLO violation rate by up to 89.6% over Legacy, because it orders trajectory tasks by deadline and chooses parallelism from estimated completion time rather than a fixed order and static configuration.

The latency metrics explain why no single policy dominates. Legacy and SRTF-SPmax reduce the service time of individual large tasks with large execution groups but sacrifice concurrency under mixed workloads, so single-rank policies can win on latency: SRTF-SP1 gives the lowest mean latency on several Qwen-Image settings, and FCFS-SP1 the lowest P95 on the A100 Qwen-Image short workload. The loss of concurrency even collapses capacity on A100 Qwen-Image foreground burst, where Legacy and SRTF-SPmax complete only 37% and 39% of submitted requests (their reported latencies are therefore optimistic). These results support the central design goal: scheduling strategy should be a policy-layer choice over one runtime substrate.

Table 2. Policy breakdown on H20 (Wan2.2 short workload).

Configuration	Added mechanism	SLO (%)	Mean (s)
Legacy (SP4)	—	27.9	471.2
FCFS-SP1	single-GPU lanes	55.7	174.8
SJF-SP1	request-size ordering	57.4	164.9
SRTF-SP1	step-level preemption	70.5	154.9
EDF-SP1	deadline ordering	83.6	143.7
EDF	dynamic parallelism	85.2	121.5

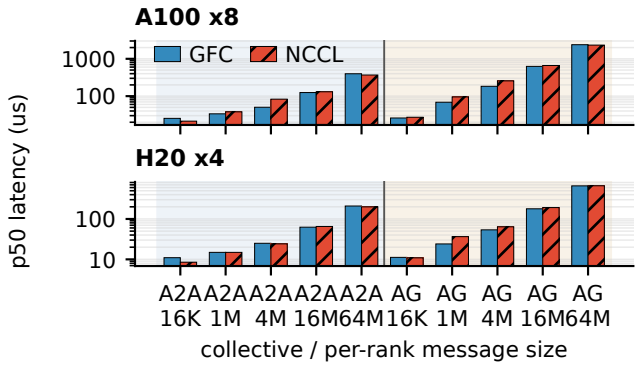


Figure 8. GFC versus NCCL p50 latency for BF16 all-to-all (A2A) and all-gather (AG) across per-rank message sizes.

6.4 Performance Breakdown

To separate the sources of Plectra's gains, Table 2 starts from Legacy and adds one mechanism per row on the H20 Wan2.2 short workload. On fixed single-GPU lanes, FCFS reaches 55.7%, and ordering whole requests by size (non-preemptive SJF) adds little. Step-level preemption (SRTF) raises attainment to 70.5% (lower than in Fig. 6, as the runs break ties among equal-work requests differently), and deadline ordering (EDF) to 83.6%. Enabling dynamic parallelism under EDF reaches 85.2% and cuts mean latency from 143.7 s to 121.5 s. The latency benefit grows when GPUs are idle: at 40% load, it cuts EDF's mean latency by 38% (81.3 s to 50.5 s) at 100% attainment, whereas full-machine SP4 reaches 70.1 s and 80%. Preemption and deadline ordering thus account for most of the SLO improvement, while elastic allocation mainly improves latency.

6.5 Runtime Overhead

To isolate the overhead of the runtime itself, we pin Plectra to FCFS-SP4, which reproduces the native Legacy configuration: the same FIFO order and a full-machine SP4 group. On the 4-GPU H20 Wan2.2 workloads, FCFS-SP4 stays within 6% of Legacy mean latency (1.06× on short, 0.95× on foreground burst) and within 2% of its throughput (0.99× and 1.02×). Policy programmability therefore introduces negligible overhead when the execution order and layout are fixed.

6.6 Group-Free Collective Performance

Fig. 8 compares steady-state GFC latency against NCCL with pre-initialized process groups. For small messages, GFC

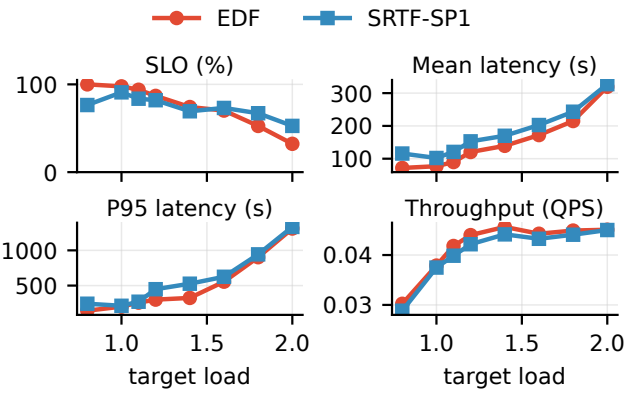


Figure 9. EDF and SRTF-SP1 under increasing arrival rate (4-GPU H20 Wan2.2 short workload); target load is normalized to estimated capacity.

can be slower because its fixed synchronization and staging overheads are not yet optimized for this regime. Diffusion-serving collectives typically use larger intermediate tensors, for which these costs are amortized; in this range, GFC generally matches or outperforms NCCL. For example, GFC reduces A100 4 MB all-to-all latency from 83.0 μs to 50.4 μs and H20 1 MB all-gather latency from 36.5 μs to 24.1 μs. Thus, GFC avoids serving-path process-group construction while maintaining competitive steady-state performance.

Scalability. Our largest testbed has 8 GPUs, so we simulate 8 to 64 GPUs in one NVL72-style symmetric-memory domain at fixed per-GPU load (Wan2.2 short workload, EDF). As the system grows, reconfigurations rise from 0.09 to 0.96 per second. Building each new group with PyTorch's default `new_group`, which synchronizes all ranks, raises mean latency over GFC by 14% at 8 GPUs and 30% at 64, while caching every group would need 70 communicators (about 36 GB) per GPU. GFC instead keeps 24 bytes of synchronization state per peer (1.5 KB at 64 GPUs).

6.7 Scaling with Arrival Rate

Fig. 9 evaluates EDF and SRTF-SP1 as arrival rate increases. At low and moderate loads, EDF achieves higher SLO attainment by prioritizing earlier deadlines and adding parallelism when needed. Under sustained overload, EDF keeps rescuing tight-deadline requests with larger sequence-parallel groups, consuming more GPUs per request and delaying new arrivals; SRTF-SP1 then wins by preserving single-rank concurrency. Thus deadline-aware parallelism helps when capacity is available but can hurt under overload.

6.8 Simulator Fidelity

Fig. 10 compares simulator predictions with real serving runs on a representative H20 workload, covering the four programmable policies from the main results plus FCFS-SP4 as a static large-group baseline. All five policies differ by at most

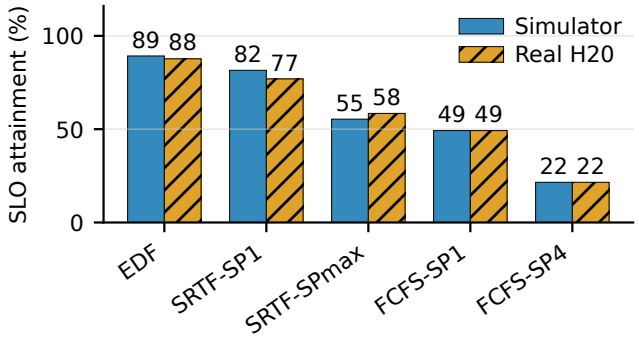


Figure 10. Simulator versus real 4-GPU H20 overall SLO attainment (Wan2.2 foreground burst); the simulator replays the exact trace and policy logic with measured stage costs.

4.7 percentage points in overall SLO attainment. The simulator is therefore accurate enough to compare policy trends and identify candidates, while final selection still requires real-system validation.

6.9 Comparison with TetriServe

We port TetriServe's scheduler to our policy interface following its released implementation: per-request optimal-shape search over 1/2/4/8-GPU splits of the remaining steps under the deadline, a step-granularity work horizon with dynamic-programming round packing, and FCFS admission truncated to the free-GPU count. In simulation, it attains 59.0%/52.3% overall SLO on the H20 Wan2.2 short and foreground-burst workloads, against 91.8%/89.2% for Plectra's EDF policy in the same simulator. TetriServe was designed for a different workload mix, and a policy tuned for one mix need not transfer to another, which is precisely why the scheduling policy should be programmable rather than fixed by the runtime.

7 Related Work

Diffusion model serving. Recent diffusion serving systems exploit stage- or step-level execution structure: TridentServe allocates resources across encoding, denoising, and decoding stages [47], TetriServe, GenServe, and DDiT schedule at step level and adapt per-request parallelism [16, 23, 49], and others target batching, mixed-resolution serving, model cascades, or adapter-intensive workloads [3, 20, 25, 39, 40]. Each realizes one particular allocation algorithm; Plectra instead provides a policy-programmable substrate in which task order, placement, and parallel degree are chosen jointly at trajectory boundaries, with group-free collectives and layout-aware migration making arbitrary scheduler-selected layouts executable. DiT inference systems provide efficient parallel execution layouts, including sequence, patch-pipeline, and CFG parallelism [10, 11, 24], which frameworks such as vLLM-Omni, SGLang Diffusion, and FastVideo integrate into deployment stacks [12, 35, 50]. Plectra is complementary: rather than proposing new parallel kernels, it enables existing layouts to be dynamically selected and reconfigured at runtime.

Collective communication and dynamic groups. Collective communication libraries such as NCCL, RCCL, UCC, and UCCL rely on statically defined communication groups [1, 29, 38, 43, 53], and recent work improves collective performance through synthesis, scheduling, and topology-aware optimization [7, 8, 17, 27, 30, 36, 45]. These systems optimize communication within a predefined group. In contrast, Plectra addresses online execution-group reconfiguration through group-free collectives, eliminating communicator construction on the serving path. Unlike sense-reversing barriers [28] and phasers [37], which keep synchronization state per barrier or phaser, GFC keeps reusable state per rank pair, so overlapping groups need no per-group state.

8 Conclusion

This paper presented Plectra, a policy-programmable runtime for elastic DiT serving. By treating GPU parallelism as a first-class schedulable resource, Plectra enables online adaptation of resource allocation to workload demands, system conditions, and service objectives. Through an asynchronous execution abstraction, group-free collectives, and a programmable runtime substrate, Plectra makes elastic parallelism practical for real-world DiT serving workloads, substantially improving throughput, latency, and SLO attainment over fixed-pipeline execution with static parallelism. More broadly, we believe that future generative AI serving systems should move beyond static resource allocation and embrace runtime-managed parallelism as a fundamental systems abstraction.

Acknowledgments

Generative AI assisted with system implementation and language polishing; the research ideas, system design, and conclusions are the authors' own.

References

[1] Advanced Micro Devices, Inc. 2026. ROCm Communication Collectives Library (RCCL). https://rocm.docs.amd.com/projects/rccl/en/latest/. Documentation.

[2] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 7, 18 pages.

[3] Sohaib Ahmad, Qizheng Yang, Haoliang Wang, Ramesh K. Sitaraman, and Hui Guan. 2025. DiffServe: Efficiently Serving Text-to-Image Diffusion Models with Query-Aware Model Scaling. arXiv:2411.15381 [cs.DC] https://arxiv.org/abs/2411.15381

[4] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 929–947. doi:10.1145/3620665.3640366

[5] Black Forest Labs. 2024. FLUX.1. https://github.com/black-forest-labs/flux. Software.

[6] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. 2024. Video generation models as world simulators. (2024). https://openai.com/research/video-generation-models-as-world-simulators

[7] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. 2021. Synthesizing optimal collective algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '21)*. ACM, 62–75. doi:10.1145/3437801.3441620

[8] DeepSeek-AI. 2025. DeepEP: An Efficient Expert-Parallel Communication Library. https://github.com/deepseek-ai/DeepEP. Software library.

[9] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, Dustin Podell, Tim Dockhorn, Zion English, Kyle Lacey, Alex Goodwin, Yannik Marek, and Robin Rombach. 2024. Scaling Rectified Flow Transformers for High-Resolution Image Synthesis. arXiv:2403.03206 [cs.CV] https://arxiv.org/abs/2403.03206

[10] Jiarui Fang, Jinzhe Pan, Aoyu Li, Xibo Sun, and Jiannan Wang. 2025. PipeFusion: Patch-level Pipeline Parallelism for Diffusion Transformers Inference. arXiv:2405.14430 [cs.CV] https://arxiv.org/abs/2405.14430

[11] Jiarui Fang, Jinzhe Pan, Xibo Sun, Aoyu Li, and Jiannan Wang. 2024. xDiT: an Inference Engine for Diffusion Transformers (DiTs) with Massive Parallelism. arXiv:2411.01738 [cs.DC] https://arxiv.org/abs/2411.01738

[12] FastVideo Team. 2025. FastVideo: A Unified Inference and Post-Training Framework for Accelerated Video Generation. https://github.com/hao-ai-lab/FastVideo. Software.

[13] Yu Gao, Haoyuan Guo, Tuyen Hoang, Weilin Huang, Lu Jiang, Fangyuan Kong, Huixia Li, Jiashi Li, Liang Li, Xiaojie Li, Xunsong Li, Yifu Li, Shanchuan Lin, Zhijie Lin, Jiawei Liu, Shu Liu, Xiaonan Nie, Zhiwu Qing, Yuxi Ren, Li Sun, Zhi Tian, Rui Wang, Sen Wang, Guoqiang Wei, Guohong Wu, Jie Wu, Ruiqi Xia, Fei Xiao, Xuefeng Xiao, Jiangqiao Yan, Ceyuan Yang, Jianchao Yang, Runkai Yang, Tao Yang, Yihang Yang, Zilyu Ye, Xuejiao Zeng, Yan Zeng, Heng Zhang, Yang Zhao, Xiaozheng Zheng, Peihao Zhu, Jiaxin Zou, and Feilong Zuo. 2025. Seedance 1.0: Exploring the Boundaries of Video Generation Models. arXiv:2506.09113 [cs.CV] https://arxiv.org/abs/2506.09113

[14] Agrim Gupta, Lijun Yu, Kihyuk Sohn, Xiuye Gu, Meera Hahn, Li Fei-Fei, Irfan Essa, Lu Jiang, and José Lezama. 2023. Photorealistic Video Generation with Diffusion Models. arXiv:2312.06662 [cs.CV] https://arxiv.org/abs/2312.06662

[15] Yoav HaCohen, Nisan Chiprut, Benny Brazowski, Daniel Shalem, Dudu Moshe, Eitan Richardson, Eran Levin, Guy Shiran, Nir Zabari, Ori Gordon, Poriya Panet, Sapir Weissbuch, Victor Kulikov, Yaki Bitterman, Zeev Melumian, and Ofir Bibi. 2024. LTX-Video: Realtime Video Latent Diffusion. arXiv:2501.00103 [cs.CV] https://arxiv.org/abs/2501.00103

[16] Heyang Huang, Cunchen Hu, Jiaqi Zhu, Ziyuan Gao, Liangliang Xu, Yizhou Shan, Yungang Bao, Sun Ninghui, Tianwei Zhang, and Sa Wang. 2025. DDiT: Dynamic Resource Allocation for Diffusion Transformer Model Serving. arXiv:2506.13497 [cs.DC] https://arxiv.org/abs/2506.13497

[17] Changho Hwang, Peng Cheng, Roshan Dathathri, Abhinav Jangda, Saeed Maleki, Madan Musuvathi, Olli Saarikivi, Aashaka Shah, Ziyue Yang, Binyang Li, Caio Rocha, Qinghua Zhou, Mahdieh Ghazimirsaeed, Sreevatsa Anantharamu, and Jithin Jose. 2026. MSCCL++: Rethinking GPU Communication Abstractions for AI Inference. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (USA) *(ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 1201–1215. doi:10.1145/3779212.3790188

[18] Diederik P Kingma and Max Welling. 2022. Auto-Encoding Variational Bayes. arXiv:1312.6114 [stat.ML] https://arxiv.org/abs/1312.6114

[19] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, Kathrina Wu, Qin Lin, Junkun Yuan, Yanxin Long, Aladdin Wang, Andong Wang, Changlin Li, Duojun Huang, Fang Yang, Hao Tan, Hongmei Wang, Jacob Song, Jiawang Bai, Jianbing Wu, Jinbao Xue, Joey Wang, Kai Wang, Mengyang Liu, Pengyu Li, Shuai Li, Weiyan Wang, Wenqing Yu, Xinchi Deng, Yang Li, Yi Chen, Yutao Cui, Yuanbo Peng, Zhentao Yu, Zhiyu He, Zhiyong Xu, Zixiang Zhou, Zunnan Xu, Yangyu Tao, Qinglin Lu, Songtao Liu, Dax Zhou, Hongfa Wang, Yong Yang, Di Wang, Yuhong Liu, Jie Jiang, and Caesar Zhong. 2025. HunyuanVideo: A Systematic Framework For Large Video Generative Models. arXiv:2412.03603 [cs.CV] https://arxiv.org/abs/2412.03603

[20] Suyi Li, Lingyun Yang, Xiaoxiao Jiang, Hanfeng Lu, Dakai An, Zhipeng Di, Weiyi Lu, Jiawei Chen, Kan Liu, Yinghao Yu, Tao Lan, Guodong Yang, Lin Qu, Liping Zhang, and Wei Wang. 2025. KATZ: efficient workflow serving for diffusion models with many adapters. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference* (Boston, MA, USA) *(USENIX ATC '25)*. USENIX Association, USA, Article 61, 16 pages.

[21] C. L. Liu and James W. Layland. 1973. Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment. *J. ACM* 20, 1 (Jan. 1973), 46–61. doi:10.1145/321738.321743

[22] Haoyu Lu, Guoxing Yang, Nanyi Fei, Yuqi Huo, Zhiwu Lu, Ping Luo, and Mingyu Ding. 2023. VDT: General-purpose Video Diffusion Transformers via Mask Modeling. arXiv:2305.13311 [cs.CV] https://arxiv.org/abs/2305.13311

[23] Runyu Lu, Shiqi He, Wenxuan Tan, Shenggui Li, Ruofan Wu, Jeff J. Ma, Ang Chen, and Mosharaf Chowdhury. 2026. TetriServe: Efficiently

Serving Mixed DiT Workloads. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Pittsburgh, PA, USA) *(ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 1982–1997. doi:10.1145/3779212.3790233

[24] Jiajun Luo, Yicheng Xiao, Jianru Xu, Yangxiu You, Rongwei Lu, Chen Tang, Jingyan Jiang, and Zhi Wang. 2025. Accelerating Parallel Diffusion Model Serving with Residual Compression. arXiv:2507.17511 [cs.CV] https://arxiv.org/abs/2507.17511

[25] Michael Luo, Aaron Hao, Zhengxu Yan, Chengkun Cao, and Quang Luong Nhat Nguyen. 2026. DiT-Serve: An Efficient Serving Engine for Diffusion Transformers. https://openreview.net/forum?id=NGNRc7rZBg

[26] Xin Ma, Yaohui Wang, Xinyuan Chen, Gengyun Jia, Ziwei Liu, Yuan-Fang Li, Cunjian Chen, and Yu Qiao. 2025. Latte: Latent Diffusion Transformer for Video Generation. arXiv:2401.03048 [cs.CV] https://arxiv.org/abs/2401.03048

[27] Ziming Mao, Yihan Zhang, Chihan Cui, Zhen Huang, Kaichao You, Zhongjie Chen, Zhiying Xu, Zhenyu Gu, Scott Shenker, Costin Raiciu, Yang Zhou, and Ion Stoica. 2026. UCCL-EP: Portable Expert-Parallel Communication. arXiv:2512.19849 [cs.DC] https://arxiv.org/abs/2512.19849

[28] John M. Mellor-Crummey and Michael L. Scott. 1991. Algorithms for scalable synchronization on shared-memory multiprocessors. *ACM Trans. Comput. Syst.* 9, 1 (Feb. 1991), 21–65. doi:10.1145/103727.103729

[29] NVIDIA Corporation. 2026. NVIDIA Collective Communication Library (NCCL). https://developer.nvidia.com/nccl. Software library.

[30] Lichen Pan, Juncheng Liu, Yongquan Fu, Jinhui Yuan, Rongkai Zhang, Pengze Li, and Zhen Xiao. 2025. Comprehensive Deadlock Prevention for GPU Collective Communication. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. ACM, 541–557. doi:10.1145/3689031.3717466

[31] William Peebles and Saining Xie. 2023. Scalable Diffusion Models with Transformers. arXiv:2212.09748 [cs.CV] https://arxiv.org/abs/2212.09748

[32] Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, David Yan, Dhruv Choudhary, Dingkang Wang, Geet Sethi, Guan Pang, Haoyu Ma, Ishan Misra, Ji Hou, Jialiang Wang, Kiran Jagadeesh, Kunpeng Li, Luxin Zhang, Mannat Singh, Mary Williamson, Matt Le, Matthew Yu, Mitesh Kumar Singh, Peizhao Zhang, Peter Vajda, Quentin Duval, Rohit Girdhar, Roshan Sumbaly, Sai Saketh Rambhatla, Sam Tsai, Samaneh Azadi, Samyak Datta, Sanyuan Chen, Sean Bell, Sharadh Ramaswamy, Shelly Sheynin, Siddharth Bhattacharya, Simran Motwani, Tao Xu, Tianhe Li, Tingbo Hou, Wei-Ning Hsu, Xi Yin, Xiaoliang Dai, Yaniv Taigman, Yaqiao Luo, Yen-Cheng Liu, Yi-Chiao Wu, Yue Zhao, Yuval Kirstain, Zecheng He, Zijian He, Albert Pumarola, Ali Thabet, Artsiom Sanakoyeu, Arun Mallya, Baishan Guo, Boris Araya, Breena Kerr, Carleigh Wood, Ce Liu, Cen Peng, Dimitry Vengertsev, Edgar Schonfeld, Elliot Blanchard, Felix Juefei-Xu, Fraylie Nord, Jeff Liang, John Hoffman, Jonas Kohler, Kaolin Fire, Karthik Sivakumar, Lawrence Chen, Licheng Yu, Luya Gao, Markos Georgopoulos, Rashel Moritz, Sara K. Sampson, Shikai Li, Simone Parmeggiani, Steve Fine, Tara Fowler, Vladan Petrovic, and Yuming Du. 2025. Movie Gen: A Cast of Media Foundation Models. arXiv:2410.13720 [cs.CV] https://arxiv.org/abs/2410.13720

[33] PyTorch Contributors. 2025. PyTorch Symmetric Memory. https://docs.pytorch.org/docs/stable/symmetric_memory.html. Accessed: 2026-06-08.

[34] Linus Schrage. 1968. Letter to the Editor—A Proof of the Optimality of the Shortest Remaining Processing Time Discipline. *Oper. Res.* 16, 3 (June 1968), 687–690. doi:10.1287/opre.16.3.687

[35] SGLang Diffusion Team. 2025. SGLang Diffusion: Accelerating Video and Image Generation. https://www.lmsys.org/blog/2025-11-07-sglang-diffusion/. LMSYS Blog.

[36] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 593–612. https://www.usenix.org/conference/nsdi23/presentation/shah

[37] Jun Shirako, David M. Peixotto, Vivek Sarkar, and William N. Scherer. 2008. Phasers: a unified deadlock-free construct for collective and point-to-point synchronization. In *Proceedings of the 22nd Annual International Conference on Supercomputing* (Island of Kos, Greece) *(ICS '08)*. Association for Computing Machinery, New York, NY, USA, 277–288. doi:10.1145/1375527.1375568

[38] Min Si, Pavan Balaji, Yongzhou Chen, Ching-Hsiang Chu, Adi Gangidi, Saif Hasan, Subodh Iyengar, Dan Johnson, Bingzhe Liu, Regina Ren, Deep Shah, Ashmitha Jeevaraj Shetty, Greg Steinbrecher, Yulun Wang, Bruce Wu, Xinfeng Xie, Jingyi Yang, Mingran Yang, Kenny Yu, Minlan Yu, Cen Zhao, Wes Bland, Denis Boyda, Suman Gumudavelli, Prashanth Kannan, Cristian Lumezanu, Rui Miao, Zhe Qu, Venkat Ramesh, Maxim Samoylov, Jan Seidel, Srikanth Sundaresan, Feng Tian, Qiye Tan, Shuqiang Zhang, Yimeng Zhao, Shengbao Zheng, Art Zhu, and Hongyi Zeng. 2026. Collective Communication for 100k+ GPUs. arXiv:2510.20171 [cs.DC] https://arxiv.org/abs/2510.20171

[39] Desen Sun, Zepeng Zhao, and Yuke Wang. 2025. PATCHEDSERVE: A Patch Management Framework for SLO-Optimized Hybrid Resolution Diffusion Serving. arXiv:2501.09253 [cs.DC] https://arxiv.org/abs/2501.09253

[40] Desen Sun, Zepeng Zhao, and Yuke Wang. 2026. MixFusion: A Patch-Level Parallel Serving System for Mixed-Resolution Diffusion Models. In *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming* (Sydney, NSW, Australia) *(PPoPP '26)*. Association for Computing Machinery, New York, NY, USA, 522–536. doi:10.1145/3774934.3786420

[41] Meituan LongCat Team, Xunliang Cai, Qilong Huang, Zhuoliang Kang, Hongyu Li, Shijun Liang, Liya Ma, Siyu Ren, Xiaoming Wei, Rixu Xie, and Tong Zhang. 2025. LongCat-Video Technical Report. arXiv:2510.22200 [cs.CV] https://arxiv.org/abs/2510.22200

[42] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (Phoenix, AZ, USA) *(MAPL 2019)*. Association for Computing Machinery, New York, NY, USA, 10–19. doi:10.1145/3315508.3329973

[43] Manjunath Gorentla Venkata, Valentine Petrov, Sergey Lebedev, Devendar Bureddy, Ferrol Aderholdt, Joshua Ladd, Gil Bloch, Mike Dubman, and Gilad Shainer. 2025. Unified Collective Communication: A Unified Library for CPU, GPU, and DPU Collectives. *IEEE Micro* 45, 2 (March 2025), 26–35. doi:10.1109/MM.2025.3534638

[44] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, Jingfeng Zhang, Jingren Zhou, Jinkai Wang, Jixuan Chen, Kai Zhu, Kang Zhao, Keyu Yan, Lianghua Huang, Mengyang Feng, Ningyi Zhang, Pandeng Li, Pingyu Wu, Ruihang Chu, Ruili Feng, Shiwei Zhang, Siyang Sun, Tao Fang, Tianxing Wang, Tianyi Gui, Tingyu Weng, Tong Shen, Wei Lin, Wei Wang, Wei Wang, Wenmeng Zhou, Wente Wang, Wenting Shen, Wenyuan Yu, Xianzhong Shi, Xiaoming Huang, Xin Xu, Yan Kou, Yangyu Lv, Yifei Li, Yijing Liu, Yiming Wang, Yingya Zhang, Yitong Huang, Yong Li, You Wu, Yu Liu, Yulin Pan, Yun Zheng, Yuntao Hong, Yupeng Shi, Yutong Feng, Zeyinzi Jiang, Zhen Han, Zhi-Fan Wu, and Ziyu Liu. 2025. Wan: Open and Advanced Large-Scale Video Generative Models. arXiv:2503.20314 [cs.CV] https://arxiv.org/abs/2503.20314

[45] William Won, Midhilesh Elavazhagan, Sudarshan Srinivasan, Swati Gupta, and Tushar Krishna. 2024. TACOS: Topology-Aware Collective Algorithm Synthesizer for Distributed Machine Learning. In *Proceedings of the 2024 57th IEEE/ACM International Symposium on Microarchitecture* (Austin, TX, USA) *(MICRO '24)*. IEEE Press, 856–870. doi:10.1109/MICRO61859.2024.00068

[46] Chenfei Wu, Jiahao Li, Jingren Zhou, Junyang Lin, Kaiyuan Gao, Kun Yan, Sheng ming Yin, Shuai Bai, Xiao Xu, Yilei Chen, Yuxiang Chen, Zecheng Tang, Zekai Zhang, Zhengyi Wang, An Yang, Bowen Yu, Chen Cheng, Dayiheng Liu, Deqing Li, Hang Zhang, Hao Meng, Hu Wei, Jingyuan Ni, Kai Chen, Kuan Cao, Liang Peng, Lin Qu, Minggang Wu, Peng Wang, Shuting Yu, Tingkun Wen, Wensen Feng, Xiaoxiao Xu, Yi Wang, Yichang Zhang, Yongqiang Zhu, Yujia Wu, Yuxuan Cai, and Zenan Liu. 2025. Qwen-Image Technical Report. arXiv:2508.02324 [cs.CV] https://arxiv.org/abs/2508.02324

[47] Yifei Xia, Fangcheng Fu, Hao Yuan, Hanke Zhang, Xupeng Miao, Yijun Liu, Suhan Ling, Jie Jiang, and Bin Cui. 2025. TridentServe: A Stage-level Serving System for Diffusion Pipelines. arXiv:2510.02838 [cs.DC] https://arxiv.org/abs/2510.02838

[48] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, Da Yin, Yuxuan Zhang, Weihan Wang, Yean Cheng, Bin Xu, Xiaotao Gu, Yuxiao Dong, and Jie Tang. 2025. CogVideoX: Text-to-Video Diffusion Models with An Expert Transformer. arXiv:2408.06072 [cs.CV] https://arxiv.org/abs/2408.06072

[49] Fanjiang Ye, Zhangke Li, Xinrui Zhong, Ethan Ma, Russell Chen, Kaijian Wang, Jingwei Zuo, Desen Sun, Ye Cao, Triston Cao, Myungjin Lee, Arvind Krishnamurthy, and Yuke Wang. 2026. GENSERVE: Efficient Co-Serving of Heterogeneous Diffusion Model Workloads. arXiv:2604.04335 [cs.DC] https://arxiv.org/abs/2604.04335

[50] Peiqi Yin, Jiangyun Zhu, Han Gao, Chenguang Zheng, Yongxiang Huang, Taichang Zhou, Ruirui Yang, Weizhi Liu, Weiqing Chen, Canlin Guo, Didan Deng, Zifeng Mo, Cong Wang, James Cheng, Roger Wang, and Hongsheng Liu. 2026. vLLM-Omni: Fully Disaggregated Serving for Any-to-Any Multimodal Models. arXiv:2602.02204 [cs.DC] https://arxiv.org/abs/2602.02204

[51] Zangwei Zheng, Xiangyu Peng, Yuxuan Lou, Chenhui Shen, Tom Young, Xinying Guo, Binluo Wang, Hang Xu, Hongxin Liu, Mingyan Jiang, Wenjun Li, Yuhui Wang, Anbang Ye, Gang Ren, Qianran Ma, Wanying Liang, Xiang Lian, Xiwen Wu, Yuting Zhong, Zhuangyan Li, Chaoyu Gong, Guojun Lei, Leijun Cheng, Limin Zhang, Minghao Li, Ruijie Zhang, Silan Hu, Shijie Huang, Xiaokang Wang, Yuanheng Zhao, Yuqi Wang, Ziang Wei, and Yang You. 2026. Open-Sora 2.0: Training a Commercial-Level Video Generation Model in $200k. arXiv:2503.09642 [cs.GR] https://arxiv.org/abs/2503.09642

[52] Zangwei Zheng, Xiangyu Peng, Tianji Yang, Chenhui Shen, Shenggui Li, Hongxin Liu, Yukun Zhou, Tianyi Li, and Yang You. 2024. Open-Sora: Democratizing Efficient Video Production for All. arXiv:2412.20404 [cs.CV] https://arxiv.org/abs/2412.20404

[53] Yang Zhou, Zhongjie Chen, Ziming Mao, ChonLam Lao, Shuo Yang, Pravein Govindan Kannan, Jiaqi Gao, Yilong Zhao, Yongji Wu, Kaichao You, Fengyuan Ren, Zhiying Xu, Costin Raiciu, and Ion Stoica. 2025. An Extensible Software Transport Layer for GPU Networking. arXiv:2504.17307 [cs.NI] https://arxiv.org/abs/2504.17307

[54] Yuan Zhou, Qiuyue Wang, Yuxuan Cai, and Huan Yang. 2024. Allegro: Open the Black Box of Commercial-Level Video Generation Model. arXiv:2410.15458 [cs.CV] https://arxiv.org/abs/2410.15458